KRPCLAB · BOOSTERLAND

How the booster lander works

Bringing a spent first stage back to the KSC launch pad, Super Heavy style, from a kRPC script

The booster separates at about 20-35 km, still moving downrange at several hundred metres per second. The autopilot flips it, burns back toward the pad, falls through the atmosphere steering with its body and its airbrakes, and lights the engines once more for a suicide burn that puts the legs on the pad.

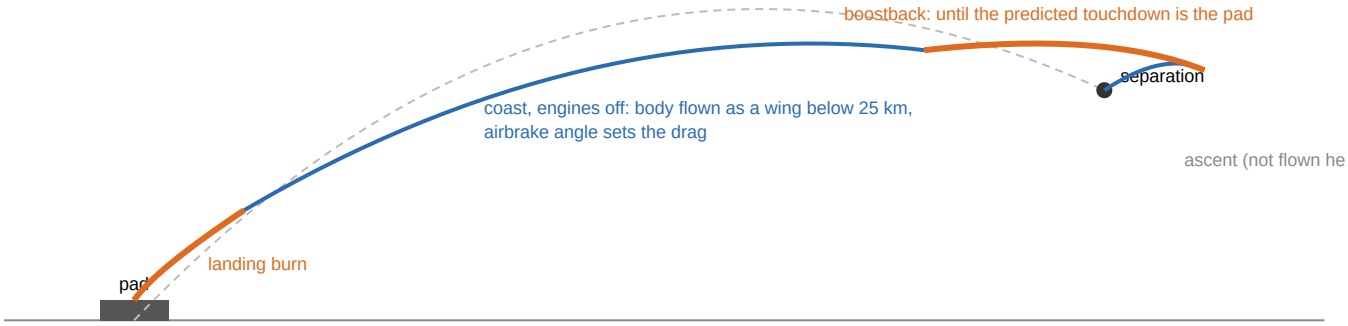


Figure 1. The flight profile, not to scale. Orange is engine-on, blue unpowered. Every part of it is re-decided each tick from a fresh prediction of where the booster will touch down.

The one idea: re-predict the landing every tick

There is no stored plan. About twenty times a second the autopilot reads the booster's position and velocity and integrates the rest of the flight to the ground in local Python: gravity, drag, body lift, the planet's rotation and the landing burn itself. Every decision that tick (where to point, how much throttle, when boostback ends, when the landing burn lights, how far the airbrakes open) is taken from that fresh prediction, so a disturbance is corrected without anything having to detect it as one.

`autoland.py` is the phase machine and the only code that talks to the game. `trajectory.py` is the propagator, `guidance.py` holds the control laws as pure functions, `engines.py` the throttle allocation, and `environment.py` keeps the world data the propagator needs (density, speed of sound, a drag curve, a lift slope, the holdable angle), cached so the integration makes no remote calls.

- It flies the landing burn too: the prediction coasts to where the suicide burn will start and keeps integrating with the engines on, through the same throttle law the vehicle flies (`trajectory.landing_command` is shared by both).
- RK4. Drag goes as v^2 through a density that changes by a factor of e every 5 km; a first-order step would bias the touchdown.
- Drag is a curve against Mach, probed with kRPC's `simulate_aerodynamic_force_at` on the vessel as it is now, deploy angle included. The curve is re-swept every 10 s once the booster falls nose-first, and 2 s after the airbrakes move.
- Lift is credited only up to what the booster can hold, and discounted by the measured aoa tracking (held ÷ commanded, steady samples only).
- Heights are measured from the feet, terrain under the legs clamped against the pad's radius; the rotation vector is measured from the vessel's velocity in two frames.

The flight, phase by phase

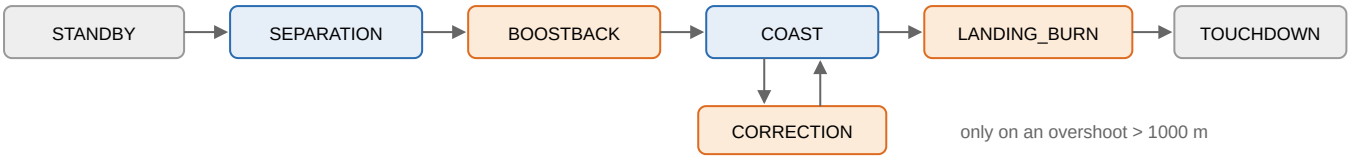
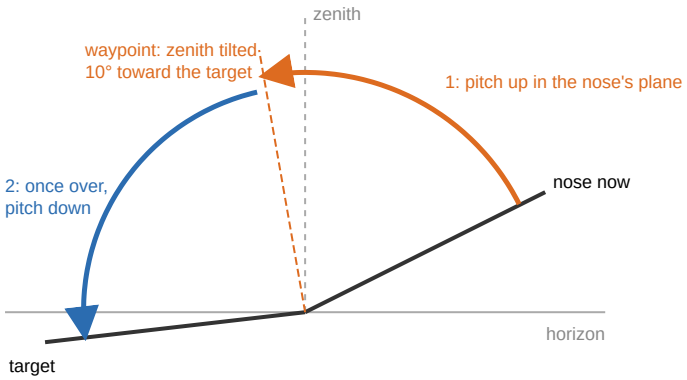


Figure 2. The phase machine. Orange phases burn the engines. CORRECTION is a short relight that COAST can enter and return from. Any phase ends the flight if the vessel is destroyed.

STANDBY waits for START	On START: RCS permitted; every airbrake deployed at 60°, the middle of the range the deploy law uses; every liquid engine activated and grouped for allocation; kRPC's roll threshold set to 180° so roll is controlled at any pointing error; the autopilot engaged on the current attitude.
SEPARATION a few seconds	`proximity.py` wraps each nearby craft in a sphere and marches the booster's motion forward against each, coasting and at full thrust along the boostback aim. The flip starts on the first tick nothing is in the way; only getting closer counts as a conflict; a 5 s backstop flips anyway.
BOOSTBACK flip and burn	Aim opposite the predicted horizontal miss and go to full throttle before the turn is finished, so the gimbal does the turning; the flip is two pitches through the vertical (below). Mid-flip the thrust is along the nose, not the aim, so while full thrust along the nose as it points now would close on a neighbour (the stage just dropped, at first) the throttle is held at zero. Throttle tapers with the remaining Δv; the burn ends when the predicted touchdown, landing burn included, has stayed within 190 m of the pad for four ticks.
COAST engines off	Flip to the descent attitude (through the vertical again) and fall back toward the pad. Below 25 km the body is flown as a wing; for the whole coast the airbrakes' deploy angle is stepped against the along-track miss (next section). At the predicted ignition point → LANDING_BURN. On a predicted overshoot over 1000 m with height and time left → CORRECTION.
CORRECTION relight, rare	A throttled-down (≤ 40%) relight, steered within 50° of retrograde, which in the game can only shorten the flight. So it is entered only when the miss is long (or mostly cross-track), never on a short miss, and only above 1000 m. Each burn is gated on a propagated gradient (metres closed per m/s), ends under 15 m, and is cut if the miss grows past 1.3 × (or 60 m beyond) the best it has reached. At most three burns, 20 s apart. Stuck off-aim and not turning, it lights at 25% so the gimbal can turn it.
LANDING_BURN suicide burn	On entry the engine gimbals stop answering the roll input. Gear down on the first tick; the RCS valve is held open for the whole burn. Throttle follows a constant-deceleration descent profile (below). The burn steers with ZEM/ZEV terminal guidance on the propagator's predicted touchdown; below 250 m it only nulls sideways speed, its attitude reference being retrograde held inside a 15° cone round vertical (vertical when ascending), and the command is rate-limited (below).
TOUCHDOWN hand back	Engines cut on reported contact (or a hover a few metres up). Every exit path runs the same shutdown: throttle zero, autopilot off, SAS on, every engine limit back to 100%, gimbal roll re-enabled, panel removed.

Attitude: flips through the vertical, roll held all flight



roof carried through the turn

The roof is held in the trajectory plane, on the side the last roof lands on when it is rotated with the nose (transport_roof). A flip over the top turns a roof that pointed up into one that points down: pitch alone, no half roll.

who rolls the booster

RCS and wheels always; the engine gimbals until the landing burn, then pitch/yaw only. kRPC credits a cluster's gimbals with almost no roll torque, so its roll gains are 1e3-1e5: fine at full thrust, a spin-up at ignition.

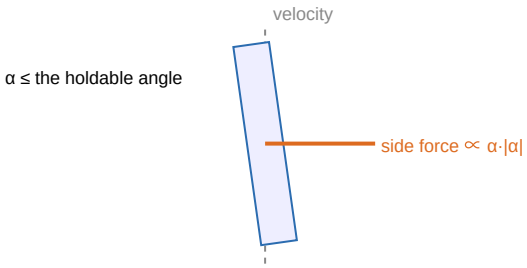
Figure 3. Any turn over 90° is two pitches through the vertical. While the nose's horizontal part points away from the target's, the command is the zenith tilted 10° toward the target; the moment it points the target's way, the command is the target. Memoryless: it re-reads the real nose every tick.

Roll reference. `guidance.roll_reference` puts the vehicle's pitch axis normal to the trajectory plane, the plane every steering command lies in. The first roof is taken on the side of that plane nearest the roof the vehicle actually has, so it never asks for more than 90° of roll. After that, each tick's roof is the side nearest the previous roof carried by the nose's own rotation from the last command to this one (`transport_roof`; near-opposite noses rotate about the trajectory normal). Below 5 m/s of horizontal speed the plane is just the last few m/s of drift, so the roof is carried rather than re-taken. kRPC's `roll_threshold` is 180°, so roll is commanded at any pointing error, including mid-flip.

Gimbal roll. `ModuleGimbal.enableRoll` is on from START, so the off-axis engines roll the booster during boostback; it is switched off when LANDING_BURN is entered, and back on at shutdown. A single on-axis engine has no roll arm either way.

Two controls in the coast: lift and drag

Engine burns after boostback can only shorten the flight. The air can push both ways, and the coast uses it twice: the angle of attack makes a side force, and the airbrakes' deploy angle sets the drag. Drag carries the range; lift trims what is left and the cross-track.



Airbrakes, 30-90°, start 60°

more open: more drag, lands shorter

every 10 s, the whole coast:

- long = along-track miss of the unsteered prediction
- S = Δlong / Δln(CdA) (two propagations)
- k = Δln(CdA) / Δangle (curve before / after a step)
- Δangle = −0.7 · long / (S·k), ≤ 20° a step
- deadband 30 m; re-sweep drag 2 s after each step

Figure 4. Left: tilting a few degrees off the velocity turns the airflow into a side force, never more than the booster can hold. Right: the deploy angle is a range control in both directions; the drag probe simulates the new shape, so the next prediction already flies it.

Drag (the whole coast). `Autoland.run_deploy` takes a Newton step every 10 s on a gain it works out for itself, so no constant belongs to one craft. The miss it answers is the along-track miss of the *unsteered* prediction: the lift solve re-nulls the steered prediction every tick, so judged on that the brakes would see ~0 until the lift saturated. S comes from flying the propagator twice, the second time with the drag curve scaled by 1.05; k comes from the drag curve swept 2 s after a step against the curve before it, median over the Mach bins; the first step is a fixed 10° that takes the first reading. The step is −0.7·long/(S·k), rate-limited to 20°, within 30-90°, with long measured along the horizontal velocity.

The holdable angle. Every 2 s `Environment.refresh_holdable` asks the game how hard the air pushes back: `simulate_aerodynamic_torque_at` on the vessel turned to a few angles off retrograde, reduced to the restoring moment per pascal of dynamic pressure and stored by Mach. Against it stands the control torque: reaction wheels plus the highest RCS torque seen plus control surfaces scaled by q, times a 0.7 margin. Where the two meet is the most the booster can hold there, typically 10° at low q falling to ~1° at 50 kPa.

Lift (below 25 km). The angle is solved each tick: propagate small test angles in-plane and out-of-plane, invert the 2×2 to null downrange and cross-track, then propagate the answer and keep it only if it lands nearer than doing nothing (else halve once, else hold retrograde). The command is capped at the holdable angle (and 10°), and the propagator credits lift at each step only up to the holdable angle at that step's Mach and q.

> **Why the coast guards exist.** The wing waits for 25 km because above it the drag curve is still being re-measured nose-first. Its answer is re-propagated before it is flown, because near the ground the sensitivity collapses and a raw solve would return the maximum angle. The deploy law steps only after the drag curve has seen its last step, because until then the prediction does not contain it.

The landing burn: a descent profile

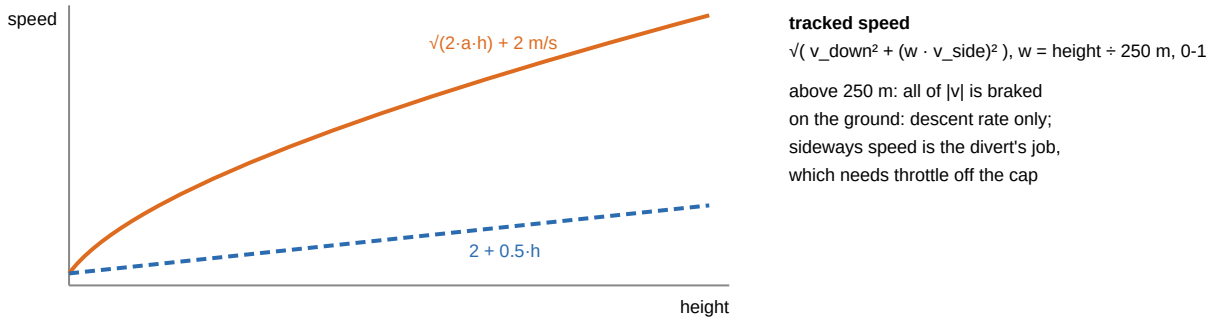


Figure 5. The throttle tracks the lower of the two curves: a constant-deceleration stop at 95% of available thrust, blending into a constant-rate final descent. Feed-forward plus a trim of 6 m/s² per m/s of error, divided by the booster's own maximum acceleration, so a 9 g and a 3.7 g booster get the same loop.

The trigger lights the burn when the height falls to the full-throttle stopping distance × 1.18 plus 1.5 s of fall. The booster then starts below the profile and the throttle stays near zero for a second or so while drag, at near-terminal velocity, does the braking. The tracked speed fades sideways speed out over the last 250 m: thrust along the vertical cannot remove it. Below 60 m an ascending booster gets zero throttle; below 12 m at under 8 m/s the throttle is capped at hover plus 0.12.

Steering. ZEM/ZEV guidance, gains 6 and 4, aimed at the propagator's predicted touchdown. The tilt is capped at 15° and at acos(throttle ÷ cap), the angle the spare throttle can pay for; the throttle is opened by 1/cos(tilt). Below 250 m the position term is dropped and only the sideways velocity is nulled, down to 4 m.

Command rate. The command the burn sends may move no faster than a quarter of a second of the vehicle's own angular acceleration per second: kRPC's credited pitch and yaw torque over the moment of inertia, the weaker axis, measured per unit throttle once a game second while the engines are lit and scaled by this tick's throttle (gimbal torque goes with thrust), never under 5°/s. A guidance command that reverses as the sideways speed changes sign becomes a ramp the vehicle can follow. Before the first torque reading the command is not limited.

Attitude controller. kRPC's attitude controller auto-tunes its gains from the torque it believes is available, and the RCS counts only while the valve is open. So the valve is held open from the burn's first tick rather than relayed on pointing error, which keeps the gains steady and the thrusters helping to hold the burn upright.

Engine allocation: one group at a time

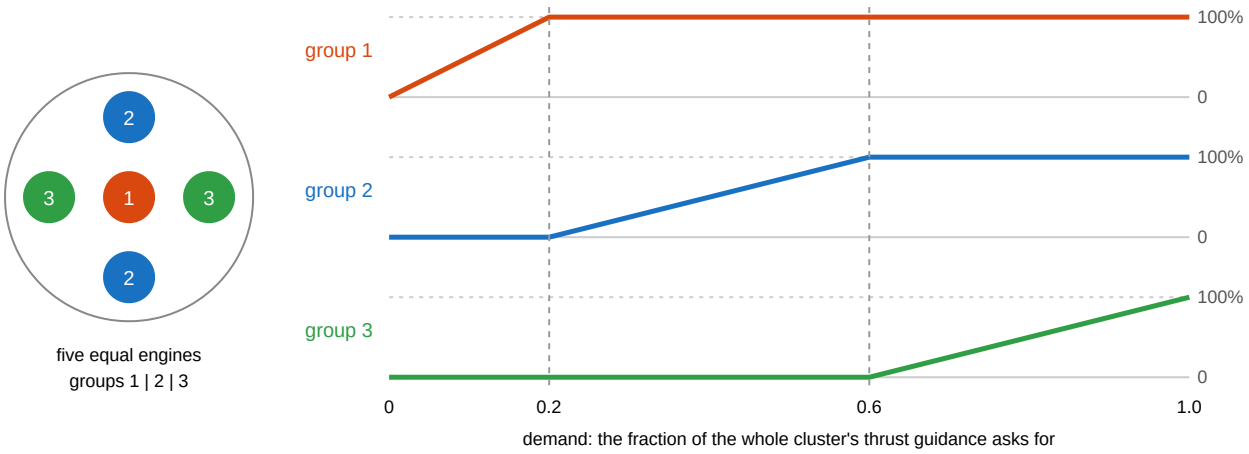


Figure 6. Each group's thrust limit against the demand. A group stays at 0 until the group before it is at 100%; only the group in progress is part-throttled. Demand is a share of the *whole* cluster, so the center engine (one fifth of it) is full at 0.2 and the first pair at 0.6. The main throttle is 1 whenever demand is above 0.

At START, `EngineCluster` reads each liquid engine's position in the vessel frame and its thrust, and `engines.group_engines` groups them about the thrust-weighted centroid: within 0.3 m of the axis is the center group; in a ring an engine is paired with the one whose offset most nearly cancels its own; whatever cannot be paired lights together, so every group's thrust passes through the axis and lighting it adds no torque. Guidance's maximum acceleration reads `max_thrust`, which ignores the limits, so the planner always sees the whole cluster. A single-engine booster is one group.

Smaller rules

- RCS: in COAST the valve opens only when the attitude controller has been saturated (input ≥ 0.95 for 0.5 s) and is not closing; in the landing burn it is held open; elsewhere it is a relay on pointing error.
- A destroyed vessel ends the flight. Its streams freeze rather than fail, so a position bit-identical for 1 game-second while moving faster than 1 m/s is a vessel lost: the flight finishes as `destroyed`. A mass drop of more than 10% in one tick is logged as parts lost.
- Every telemetry line carries the commanded nose's angle off retrograde (`cmd=`) and off vertical (`cv=`), how fast the command moves (`cw=`), the pointing error (`err=`), the largest control input (`sat=`) and each input (`in=` pitch/yaw/roll), the roll error (`roll=`, meaningful when `err` is small), the nose's swing and roll rates (`w=`, `r=`), and, for a split cluster while it can burn, each engine group's delivered thrust as a fraction of its maximum (`eng=`), beside the angle held (`hold=`) and the holdable angle (`can=`). The landing burn logs kRPC's gains and available torque at its start and under thrust.
- Every log ends with the loop rate per phase, and the config line carries a fingerprint of the defaults. Throttle divergence is logged: a commanded throttle the vessel does not report is a physical throttle axis winning.

How it is measured

Each flight writes a numbered log (`logs/LOG<n>`), telemetry paced in game time, with the settings that differ from the defaults. Tunables live only in `config.py`; `--set FIELD=VALUE` overrides one for a flight. Six reference saves (`quicksave`, `qs_hot`, `qs_cold`, `qs_steep`, `qs_north` for the 17 t single-engine booster; `qs_heavy` for the 99 t five-engine booster) give the separation states; `tools/savegen.py` writes more, and `tests/fakeksp` runs the closed loop offline for screening. `tools/brot.sh` flies arms × saves rotated over the farm at 1× (the booster has no time-scale governor).

State of the code on 2026-10-17 (defaults 71ef95a5). Tunables: `boosterland/config.py`.